

ML Algorithm used for the model- Random Forest

1. Algorithm Type & Objective

Random Forest is an ensemble learning algorithm used for both classification and regression tasks. In this project, it is used for multiclass text classification. The main objective is to improve prediction accuracy by combining the outputs of multiple decision trees.

2. How the Algorithm Works (Mathematical Intuition)

-Bootstrap Sampling – Multiple subsets of the training data are created by randomly sampling with replacement.

-Decision Tree Training – Each subset is used to train a separate decision tree.

-Random Feature Selection – At each node split, only a random subset of features is considered, which reduces overfitting.

-Voting – For classification, each tree votes for a class, and the class with the majority votes becomes the final prediction.

Mathematical intuition:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_n(x)\}$$

Where $h_i(x)$ is the prediction of the i^{th} tree and mode selects the most frequent predicted class.

3. Key Hyperparameters

-n_estimators – Number of trees in the forest (default: 100). Higher values generally improve accuracy but increase training time.

-max_depth – Maximum depth of each tree (default: None). Controls complexity; smaller values prevent overfitting.

-max_features – Number of features considered for splitting a node ('sqrt' is default for classification).

-min_samples_split – Minimum samples required to split an internal node (default: 2).

-min_samples_leaf – Minimum samples required at a leaf node (default: 1).

-class_weight – Adjusts weights for classes to handle imbalance (e.g., 'balanced').

4. Strengths & Limitations

Strengths:

-Handles imbalanced datasets well when using class_weight='balanced'.

-Can model non-linear relationships effectively.

-Less prone to overfitting than single decision trees.

-Works well with high-dimensional data such as TF-IDF vectors.

Limitations:

-Can be computationally expensive for very large datasets.

-Model size can be large due to many trees.

-Predictions are harder to interpret compared to simpler models like Logistic Regression.

5. When to Use / When Not to Use

-Use When:

The dataset is imbalanced and requires a robust classifier.

Relationships between features and labels are non-linear.

High accuracy is more important than model interpretability.

-Avoid When:

You need a very fast prediction model for real-time systems.

The dataset is small and a simpler model can perform equally well.